



廣東工業大學

GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计报告

《基于 STM32F103C8T6 的智能手表设计》

姓 名： 叶锦前

学 号： 3124009603

专 业： 微电子科学与工程

班 级： 微电子 2 班

队 名： 克笔愣组

队 友： 周瑜亮、郭浩哲

指导老师： 周贤中



集成电路学院

2026 年 7 月 11 日

摘要

本文介绍了嵌入式系统课程设计项目——基于 STM32F103C8T6 的智能手表设计。系统采用 FreeRTOS 抢占式多任务内核进行调度，集成了 1.3 寸 OLED 显示屏、MPU6050 六轴传感器、旋转编码器以及 JDY-31 经典蓝牙通信模块。项目实现了片内 RTC 精准走时、多级菜单交互、动态阈值波峰检测计步以及蓝牙无线数据双向同步等核心功能。在熄屏状态下，系统仍能维持 RTC 走时与计步中断唤醒。此外，通过软件 I2C 与硬件 I2C 的物理隔离设计，有效规避了总线死锁问题。本项目全面展示了 RTOS 在可穿戴设备多任务调度中的高效性及综合应用价值。

关键词：STM32；FreeRTOS；智能手表；计步算法；蓝牙同步

目录

1	项目简介	1
2	项目设计目标	1
3	项目设计与实现	2
3.1	硬件设计	2
3.1.1	核心控制与总线隔离	2
3.1.2	电源与开机电路	2
3.2	软件架构设计	2
3.2.1	FreeRTOS 任务规划	2
3.3	核心技术细节	3
3.3.1	动态滤波计步算法	3
3.3.2	熄屏与中断唤醒机制	3
4	功能展示	3
4.1	手表端可视化 UI 界面	3
4.2	蓝牙上位机协同操作	4
5	物料与成本	5
5.1	系统物料清单	5
6	个人贡献（叶锦前）	5
6.1	主要工作与代码实现	5
6.2	故障排查与技术攻坚	6
6.2.1	蓝牙通信“只能收不能发”问题	6
6.2.2	开机卡顿与不熄屏问题	6
7	项目成果与反思	6
7.1	成果总结	6
7.2	项目评估与反思	7
A	核心代码清单	7

1 项目简介

本项目是嵌入式系统课程设计作业，以 STM32F103C8T6 最小系统板为核心主控，采用 FreeRTOS 抢占式多任务内核进行调度 [cite: 1]。项目集成 OLED 显示、MPU6050 六轴传感器、旋转编码器、蓝牙通信模块，实现智能手表的时间显示、运动计步、人机交互、无线数据同步等完整功能 [cite: 1]。

本设计的主要功能包括：

1. **片内 RTC 精准走时：**复用主控片内 RTC 外设与板载 32.768kHz 晶振（LSE）走时 [cite: 1]。熄屏期间维持独立后备域走时，亮屏后时间不丢失 [cite: 1]。
2. **多任务实时调度：**基于 FreeRTOS 实现多任务高并发调度，彻底摒弃低效的裸机大循环轮询 [cite: 5]。
3. **人机交互系统：**通过 EC11 旋转编码器的定时器硬件正交解码功能，实现流畅的多级界面菜单切换 [cite: 1, 5]。
4. **运动计步与感知：**MPU6050 实时读取加速度与陀螺仪原始数据，并通过动态阈值波峰检测算法实现精准计步 [cite: 1]。
5. **无线数据同步：**配合自行开发的 PC 端图形化上位机，实现运动数据与时间的双向蓝牙无线同步 [cite: 1]。

2 项目设计目标

本项目完成课程大纲全部基本要求，并精准覆盖以下扩展要求 [cite: 1, 5]：

- **扩展要求 1（编码器多级菜单）：**引入 EC11 旋转编码器，利用定时器硬件正交解码，实现多级结构体菜单的流畅翻页、参数调整与按键确认 [cite: 1, 5]。
- **扩展要求 2（蓝牙上位机同步）：**配合 PC 端图形化上位机，实现运动数据与时间的双向无线同步 [cite: 1, 5]。
- **扩展要求 4（运动计步与数据记录）：**开发基于加速度计的动态阈值加波峰检测滤波算法，过滤日常杂散抖动 [cite: 1, 5]。支持步数统计、历史数据 RAM 缓存与内部 Flash 掉电持久化 [cite: 1, 5]。

- **总线隔离与高可用设计:** 采用软件 I2C 驱动 OLED 与硬件 I2C 驱动 MPU6050 相结合的双总线物理隔离方案, 从根本上解决 STM32F1 硬件 I2C 死锁冲突缺陷 [cite: 1, 4]。

3 项目设计与实现

3.1 硬件设计

3.1.1 核心控制与总线隔离

选用 STM32F103C8T6 最小系统板, 外设通信总线经过精心分配以避免冲突 [cite: 1, 4]:

- **OLED 显示模块:** 采用 1.3 寸 OLED, 接口分配为 PB10/PB11[cite: 4]。由于刷新对时序不敏感, 采用软件模拟 I2C 规避总线死锁 [cite: 4]。
- **运动传感器模块:** MPU6050 模块通过硬件 I2C1 (PB6/PB7) 连接, 保证高频传感器采样的实时吞吐 [cite: 4]。两路 I2C 物理隔离, 任务间互不干扰 [cite: 1, 4]。

3.1.2 电源与开机电路

系统由 3.7V 锂电池供电, 搭配 TP4056 充电模块与 XC6206 LDO 输出 3.3V 稳压 [cite: 1, 4]。搭配分立元件设计了一键开机拓扑: 采用 SI2301 (PMOS) 搭配 S8050 (NPN)[cite: 1, 4]。按键按下导通 PMOS 上电, 主控运行后立即拉高 PA8 维持 NPN 导通实现软自锁 [cite: 4]。

3.2 软件架构设计

3.2.1 FreeRTOS 任务规划

系统由四个核心任务并发驱动, 任务间通过消息队列与互斥信号量安全通信 [cite: 1, 5]:

1. **Task_Power (优先级 4):** 电源状态与低功耗模式管理 [cite: 1, 5]。检测到 10s 无操作后仅关闭 OLED, MCU 保持全速运行以规避 I2C 采样冲突 [cite: 1, 4]。
2. **Task_Sensor (优先级 3):** 每 20ms 定时周期唤醒, 读取 MPU6050 数据, 执行计步算法运算, 并缓存运动数据 [cite: 1, 5]。

3. **Task_UI (优先级 2):** 事件驱动型, 维护多级结构体菜单状态机, 利用软件 I2C 渲染刷新 OLED 界面 [cite: 1, 5]。
4. **Task_Comm (优先级 1):** 数据队列触发 [cite: 1]。负责数据打包、蓝牙串口异步发送及上位机指令解析 [cite: 1, 5]。

3.3 核心技术细节

3.3.1 动态滤波计步算法

系统提取 MPU6050 三轴加速度, 计算合加速度 [cite: 4]。将数据推入滑动窗口滤除高频电噪声后, 采用动态阈值法: 实时计算过去 50 个采样点的最值, 以其均值作为判定步态波峰的动态基准线 [cite: 4]。当当前加速度穿越该基准线且距离上次计步时间差大于 200ms 时 (防止单步抖动多计), 有效步数加一 [cite: 4]。

3.3.2 熄屏与中断唤醒机制

由于 MPU6050 每 20ms 读取数据时极易与 STOP 模式唤醒产生 I2C 总线竞争, 系统改为仅关屏的策略 [cite: 3]。MPU6050 配置为锁存模式, 触发 STM32 的 EXTI 外部中断 (PA4) [cite: 3]。在熄屏期间, 抬腕运动即可产生稳定的中断信号唤醒屏幕, 同时蓝牙与 RTC 走时均正常工作 [cite: 1, 3]。

4 功能展示

4.1 手表端可视化 UI 界面

基于软件 I2C 实现了底层绘图原语 (画点、线、矩形、可缩放大字号及反白绘制), 并重构了四大界面体系 [cite: 3]:

- **TIME (主时间页):** 反白标题栏, 大号 HH:MM 与小号秒钟, 底部配备秒进度条直观显示秒针流逝, 直观佐证 RTC 走时 [cite: 3]。
- **STEPS (计步统计页):** 大号今日步数, 辅以每日目标进度条及累计步数 [cite: 3]。
- **SENSOR (感知排查页):** 加速度与陀螺仪分栏显示, 配备专属的 INT: 硬件中断计数器 (用于抬腕唤醒链路自检窗口) [cite: 3]。

- **SETTING (设置调整页)**: 编辑时当前字段反白高亮, 底部附操作提示, 旋转编码器调整丝滑无卡顿 [cite: 3]。

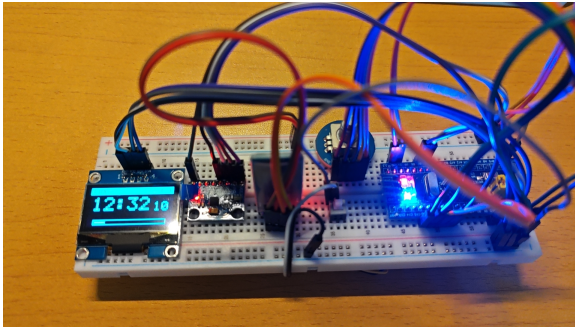


图 1 TIME (主时间界面页)

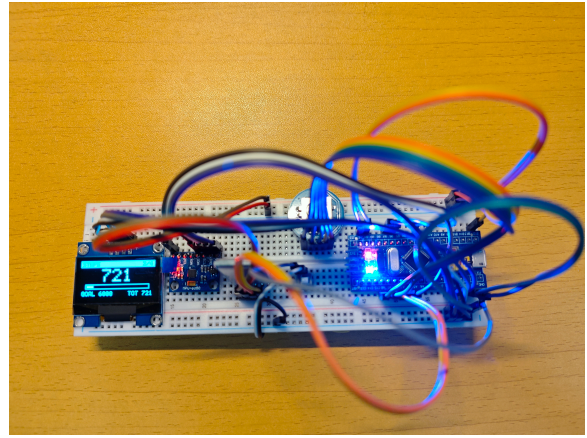


图 2 STEPS (计步统计界面页)

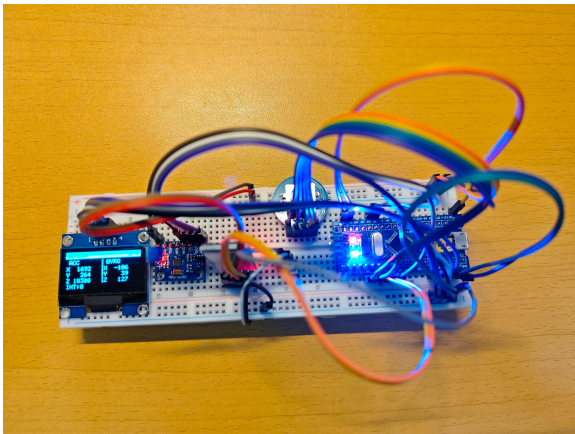


图 3 SENSOR (感知自检界面页)

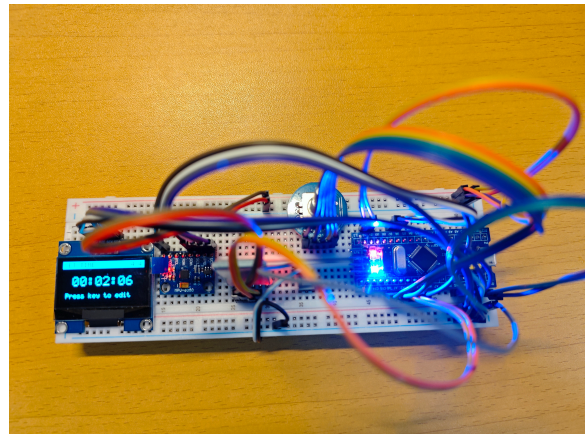


图 4 SETTING (设置调整界面页)

图 5 智能手表端四级可视化 UI 菜单实物展示

4.2 蓝牙上位机协同操作

PC 端使用 Python 与 PyQt5 开发了图形化客户端 [cite: 1, 2]。电脑配对 JDY-31 蓝牙模块后, 通过下拉框连接生成的虚拟串口, 即可实时查看手表时间、步数与加速度数据 [cite: 1]。用户还可进行校时及请求同步操作 [cite: 1]。

表 1 系统物料与采购成本清单

物料名称	规格型号	数量	单价 (元)	用途
主控核心板	STM32F103C8T6 最小系统板	1	0.00	核心控制单元
OLED 显示模块	1.3 寸 I2C 接口 (蓝光)	1	14.50	界面显示
交互器件	EC11 带微动开关 (轴长 15mm)	1	3.50	菜单切换调节
通信模块	JDY-31 带底板 (带状态指示)	1	6.50	蓝牙同步
传感器	MPU6050 传感器模块	1	6.80	六轴姿态检测
电源系统	3.7V 锂电池 +TP4056+XC6206	1 套	11.00	供电与充电管理
开机电路	SI2301(PMOS)+S8050(NPN)	1 套	2.50	开关机逻辑
原型辅料	优质面包板 + 杜邦线 + 阻容件	1 套	7.00	原型搭建调试
预算总计	-	-	-	约 51.80 元

5 物料与成本

5.1 系统物料清单

6 个人贡献（叶锦前）

在本项目中，我（叶锦前）主要负责 UI 人机交互、底层通信协议封装及图形化上位机开发工作 [cite: 1, 2]。

6.1 主要工作与代码实现

- **多级菜单状态机与 UI 驱动：**负责编写 OLED 显示驱动，并实现了多级菜单状态机 [cite: 2, 5]。
- **旋转编码器正交解码：**编写编码器驱动，确保按键微动与菜单层级跳转的稳定性 [cite: 2, 5]。
- **全栈上位机开发：**利用 Python (PyQt5) 独立编写蓝牙上位机，包含文本行协议解析以及数据帧双向交互 [cite: 1, 2]。
- **工程规范与技术文档：**作为项目主笔，负责维护《项目计划书》《系统设计文档》《中期检查报告》及故障排查记录的撰写 [cite: 1, 5]。

6.2 故障排查与技术攻坚

6.2.1 蓝牙通信“只能收不能发”问题

- **问题表现：**手机连接 JDY-31 模块后，手表现象如同只能收不能发，疑似连不上手机 [cite: 3]。
- **根因分析：**无操作 10s 后进入 STOP 模式导致 USART2 无时钟，手表既不推送状态也收不到命令 [cite: 3]。此外，HAL 库的 UART 收发共用一把锁，中断里重挂接收若撞上任务发送持锁会返回 BUSY，导致接收断链；且过载/帧错误后 HAL 不会自动恢复接收 [cite: 2, 3]。
- **解决方案：**收到任一蓝牙字节即刷新活动计时，会话期间不休眠 [cite: 2, 3]。新增 HAL_UART_ErrorCallback 清错并重挂，并在 Task_Comm 加 RX 看门狗兜底，发现接收断链就重挂 [cite: 2, 3]。

6.2.2 开机卡顿与不熄屏问题

- **问题表现：**开机偶发卡在 TIME 页约 10s 且熄屏不响应 [cite: 2, 3]。
- **根因分析：**MPU6050 运动中断每触发一次就刷新一次“活动计时”，导致微动反复触发中断，空闲计时永远清零，永不熄屏 [cite: 2, 3]。同时 PA4 原为浮空输入，接触不良悬空时引发 EXTI 中断风暴饿死任务；且较高优先级的 Task_Sensor 在 I2C 异常时空转重试，饿死 UI [cite: 2, 3]。
- **解决方案：**运动中断改为只计数、不刷新活动计时 [cite: 3]。PA4 改内部下拉杜绝浮空中断风暴，并在 Task_Sensor 连续 5 次 I2C 失败后退避 200ms [cite: 2, 3]。

7 项目成果与反思

7.1 成果总结

本项目以极低的成本，成功打造了一款基于 FreeRTOS 架构的智能手表原型 [cite: 5]。系统实现了高频 MPU6050 数据解算与精准计步，并在熄屏态下维持 RTC 持续走时与抬腕唤醒 [cite: 1, 5]。配合自主开发的 PC 端蓝牙上位机，完整打通了无线通信及数据同步链路 [cite: 5]。

7.2 项目评估与反思

- **成功之处：**彻底抛弃了低效裸机大循环开发模式，利用 FreeRTOS 抢占式内核和消息队列安全通信，消除了系统阻塞 [cite: 1, 5]。在排查抬腕不醒等问题时，积累了扎实的底层故障分析经验 [cite: 3]。
- **不足之处：**第一，由于 Task_Power 与 Task_Sensor 存在 I2C 总线访问竞争，导致在 STOP 深度休眠模式下唤醒不稳定 [cite: 3]。最终退而求其次放弃了 STOP 模式，改为仅关闭 OLED 以保持 MCU 全速运行，妥协了部分极致功耗指标 [cite: 3]。第二，计步算法在部分测试场景下存在一定的计步不准现象 [cite: 2]。这是因为为了保证抬腕唤醒中断的正常触发，团队放宽了运动检测路径的低通滤波器（DLPF 至 44Hz） [cite: 2]。这导致软件计步算法的 8 点滑动平均更容易受到日常高频抖动噪声的干扰，加之动态阈值参数固定，泛化能力仍有待提高 [cite: 2]。

A 核心代码清单

完整的固件工程（VSCode + CMake + arm-none-eabi-gcc）、上位机 Python 源码及文档资源均已归档于 GitHub 仓库中，项目拓扑涵盖 Firmware/ 与 Host/ 核心目录 [cite: 1]。